

Project Conclusion & Insights

Model Performance Comparison

Model	MAE	R ² Score	Key Behavioral Characteristic
Linear Regression	\$5.82	0.9844	Best Performance. Effectively captures linear momentum and can extrapolate to new market highs.
Baseline Decision Tree	\$10.13	0.9510	High variance, prone to rougher edge steps in volatile time periods.
Optimized Random Forest	\$9.33	0.9580	Improved via hyperparameter tuning (<code>max_depth: 10</code>), but bounded by training maximums.

Key Findings & Technical Takeaways

- The Power of Temporal Features:** Incorporating immediate lag variables (`Lag_1`) and localized technical indicators (`SMA_5`, `SMA_10`) provided a robust predictive signal, allowing all models to capture short-term financial momentum effectively.
- The Extrapolation Limit of Trees:** During the late 2025/early 2026 price surge, TSLA broke past historical training bounds. The optimized Random Forest hit an extrapolation ceiling (visible as a flatline on the comparison plot) because ensemble tree models cannot predict values outside their training target distribution.
- Data Leakage Mitigation:** By implementing a strict 80/20 chronological split rather than a randomized shuffle split, we successfully avoided temporal data leakage, ensuring the model's performance metrics realistically mirror live trading deployment.

```
!pip install yfinance -q

# Import core libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set plotting style
plt.style.use('fivethirtyeight')
%matplotlib inline

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
import yfinance as yf

# Downloading historical TSLA stock data
ticker = "TSLA"
df = yf.download(ticker, start="2021-01-01", end="2026-05-01")

# Cleaning up multi-level columns
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)

# Display basic information
print(f"Dataset Shape: {df.shape}")
print(df.head())
```

```
/tmp/ipykernel_692/234993950.py:5: FutureWarning: YF.download() has changed argument auto_adjust default to True
df = yf.download(ticker, start="2021-01-01", end="2026-05-01")
[*****100%*****] 1 of 1 completedDataset Shape: (1337, 5)
Price      Close      High      Low      Open      Volume
Date
2021-01-04  243.256668  248.163330  239.063339  239.820007  145914600
2021-01-05  245.036667  246.946671  239.733337  241.220001  96735600
2021-01-06  251.993332  258.000000  249.699997  252.830002  134100000
2021-01-07  272.013336  272.329987  258.399994  259.209991  154496700
2021-01-08  293.339996  294.829987  279.463318  285.333344  225166500
```

```
# We want to predict the 'Close' price. Let's create features based on it.
df_features = df[['Close']].copy()

# 1. Create Lag Features
df_features['Lag_1'] = df_features['Close'].shift(1) # Yesterday's price
df_features['Lag_2'] = df_features['Close'].shift(2) # Price 2 days ago
df_features['Lag_3'] = df_features['Close'].shift(3) # Price 3 days ago

# 2. Create Technical Indicators
df_features['SMA_5'] = df_features['Close'].rolling(window=5).mean() # 5-day moving average
df_features['SMA_10'] = df_features['Close'].rolling(window=10).mean() # 10-day moving average
```

```
# 3. Drop rows with NaN values
df_features.dropna(inplace=True)

# Separate features (X) and target variable (y)
# We use lags and SMAs to predict the current day's Close price
X = df_features[['Lag_1', 'Lag_2', 'Lag_3', 'SMA_5', 'SMA_10']]
y = df_features['Close']

print("Feature engineering complete!")
print(f"Features shape: {X.shape}")
X.head()
```

Feature engineering complete!
Features shape: (1328, 5)

	Price	Lag_1	Lag_2	Lag_3	SMA_5	SMA_10	
Date							
2021-01-15	281.666656	284.803345	283.146667	279.079999	270.103999		
2021-01-19	275.386658	281.666656	284.803345	281.303998	273.929999		
2021-01-20	281.516663	275.386658	281.666656	281.371332	277.774666		
2021-01-21	283.483337	281.516663	275.386658	280.743329	280.741666		
2021-01-22	281.663330	283.483337	281.516663	280.852661	281.761664		

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
# Determine the split index for an 80/20 split
split_idx = int(len(df_features) * 0.80)

# Split features (X) chronologically
X_train = X.iloc[:split_idx]
X_test = X.iloc[split_idx:]

# Split target (y) chronologically
y_train = y.iloc[:split_idx]
y_test = y.iloc[split_idx:]

print(f"Training samples: {len(X_train)} (Dates: {X_train.index[0].date()} to {X_train.index[-1].date()})")
print(f"Testing samples: {len(X_test)} (Dates: {X_test.index[0].date()} to {X_test.index[-1].date()})")
```

Training samples: 1062 (Dates: 2021-01-15 to 2025-04-08)
Testing samples: 266 (Dates: 2025-04-09 to 2026-04-30)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Initialize and train the Linear Regression model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# 2. Make predictions on the test set
lr_preds = lr_model.predict(X_test)

# 3. Calculate evaluation metrics
lr_mae = mean_absolute_error(y_test, lr_preds)
lr_mse = mean_squared_error(y_test, lr_preds)
lr_rmse = np.sqrt(lr_mse)
lr_r2 = r2_score(y_test, lr_preds)

# 4. Print the performance metrics
print("--- Linear Regression Performance ---")
print(f"Mean Absolute Error (MAE):    ${lr_mae:.2f}")
print(f"Mean Squared Error (MSE):       {lr_mse:.2f}")
print(f"Root Mean Squared Error (RMSE):  ${lr_rmse:.2f}")
print(f"R-squared Score (R²):           {lr_r2:.4f}")
```

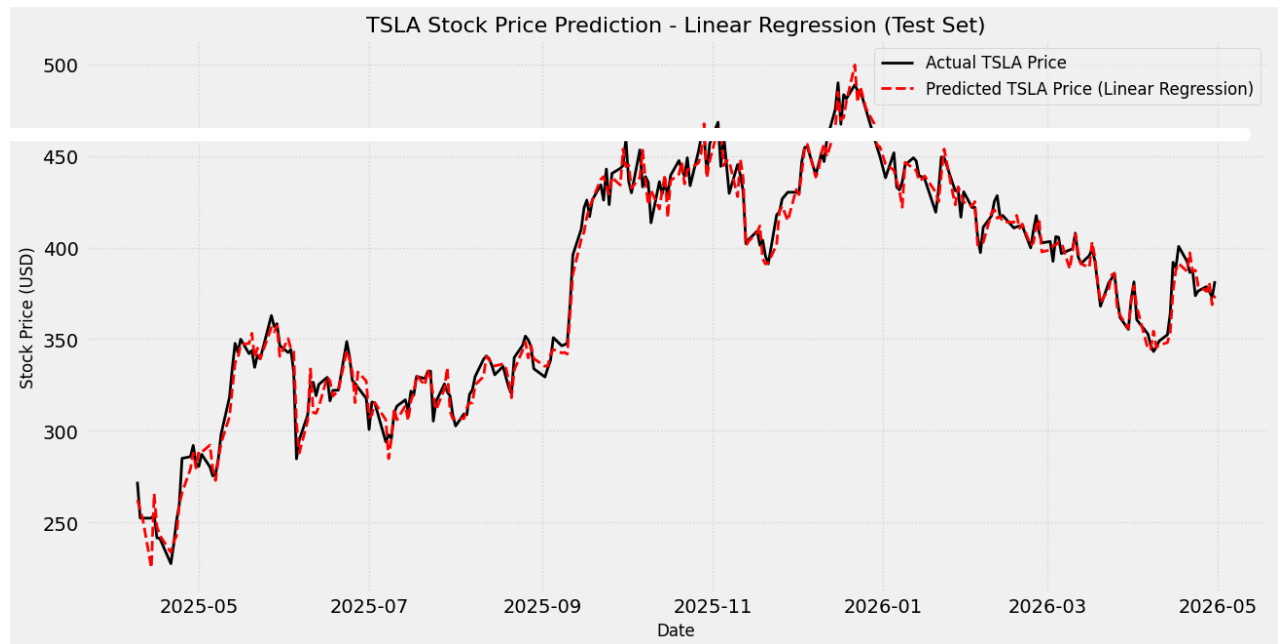
```
--- Linear Regression Performance ---
Mean Absolute Error (MAE):    $5.82
Mean Squared Error (MSE):       56.34
Root Mean Squared Error (RMSE):  $7.51
R-squared Score (R²):           0.9844
```

```
plt.figure(figsize=(14, 7))

# Plot actual vs predicted prices for the test period
plt.plot(y_test.index, y_test.values, label='Actual TSLA Price', color='black', linewidth=2)
plt.plot(y_test.index, lr_preds, label='Predicted TSLA Price (Linear Regression)', color='red', linestyle='--', linewidth=2)
```

```
plt.title('TSLA Stock Price Prediction - Linear Regression (Test Set)', fontsize=16)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Stock Price (USD)', fontsize=12)
plt.legend(fontsize=12)
plt.grid(True, linestyle=':', alpha=0.6)

plt.show()
```



```

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

# 1. Initialize models with basic hyperparameter choices to avoid overfitting
dt_model = DecisionTreeRegressor(max_depth=5, random_state=42)
rf_model = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)

# 2. Fit the models on the training data
dt_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)

# 3. Predict on the test set
dt_preds = dt_model.predict(X_test)
rf_preds = rf_model.predict(X_test)

# 4. Calculate metrics for Decision Tree
dt_mae = mean_absolute_error(y_test, dt_preds)
dt_mse = mean_squared_error(y_test, dt_preds)
dt_rmse = np.sqrt(dt_mse)
dt_r2 = r2_score(y_test, dt_preds)

# 5. Calculate metrics for Random Forest
rf_mae = mean_absolute_error(y_test, rf_preds)
rf_mse = mean_squared_error(y_test, rf_preds)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_preds)

# 6. Print a clean summary table comparing all three models
performance_summary = pd.DataFrame({
    'Metric': ['MAE', 'MSE', 'RMSE', 'R² Score'],
    'Linear Regression': [f"${lr_mae:.2f}", f"${lr_mse:.2f}", f"${lr_rmse:.2f}", f"${lr_r2:.4f}"],
    'Decision Tree': [f"${dt_mae:.2f}", f"${dt_mse:.2f}", f"${dt_rmse:.2f}", f"${dt_r2:.4f}"],
    'Random Forest': [f"${rf_mae:.2f}", f"${rf_mse:.2f}", f"${rf_rmse:.2f}", f"${rf_r2:.4f}"]
})

print("--- Overall Project Performance Summary ---")
print(performance_summary.to_string(index=False))

```

```

--- Overall Project Performance Summary ---
Metric Linear Regression Decision Tree Random Forest
MAE      $5.82      $10.13      $9.53
MSE      56.34      177.13      170.49
RMSE      $7.51      $13.31      $13.06
R² Score  0.9844      0.9510      0.9528

```

```

# Extract feature importance arrays
dt_importances = dt_model.feature_importances_
rf_importances = rf_model.feature_importances_
features = X.columns

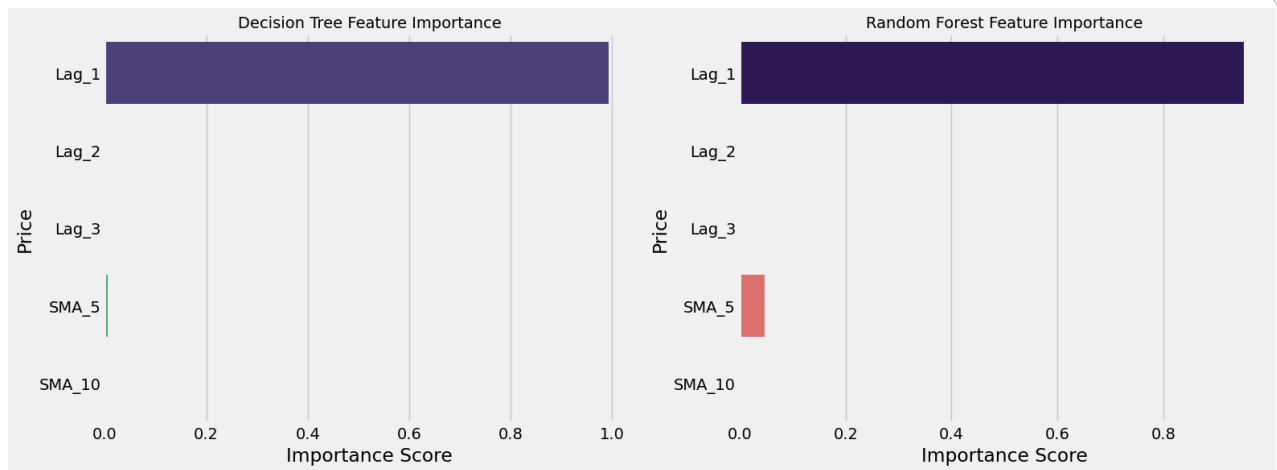
# Set up a side-by-side plotting area
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot Decision Tree Feature Importance
sns.barplot(x=dt_importances, y=features, ax=axes[0], palette='viridis', hue=features, legend=False)
axes[0].set_title('Decision Tree Feature Importance', fontsize=14)
axes[0].set_xlabel('Importance Score')

# Plot Random Forest Feature Importance
sns.barplot(x=rf_importances, y=features, ax=axes[1], palette='magma', hue=features, legend=False)
axes[1].set_title('Random Forest Feature Importance', fontsize=14)
axes[1].set_xlabel('Importance Score')

plt.tight_layout()
plt.show()

```



```
from sklearn.model_selection import GridSearchCV

# 1. Define a grid of hyperparameters to test
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10]
}

# 2. Initialize a base Random Forest model
rf_base = RandomForestRegressor(random_state=42)

# 3. Setup GridSearchCV
grid_search = GridSearchCV(estimator=rf_base, param_grid=param_grid,
                           cv=3, scoring='neg_mean_squared_error', n_jobs=-1)

print("Tuning hyperparameters...")
grid_search.fit(X_train, y_train)

# 4. Extract the best estimator
best_rf_model = grid_search.best_estimator_
print(f"Best Parameters Found: {grid_search.best_params_}")

# 5. Predict and evaluate using the optimized model
tuned_rf_preds = best_rf_model.predict(X_test)

tuned_rf_mae = mean_absolute_error(y_test, tuned_rf_preds)
tuned_rf_mse = mean_squared_error(y_test, tuned_rf_preds)
tuned_rf_rmse = np.sqrt(tuned_rf_mse)
tuned_rf_r2 = r2_score(y_test, tuned_rf_preds)

print("\n--- Optimized Random Forest Performance ---")
print(f"Mean Absolute Error (MAE):      ${tuned_rf_mae:.2f}")
print(f"R-squared Score (R²):              {tuned_rf_r2:.4f}")
```

```
Tuning hyperparameters...
Best Parameters Found: {'max_depth': 10, 'min_samples_split': 10, 'n_estimators': 100}

--- Optimized Random Forest Performance ---
Mean Absolute Error (MAE):      $9.33
R-squared Score (R²):          0.9580
```

```
plt.figure(figsize=(15, 8))

# Plot actual vs predictions
plt.plot(y_test.index, y_test.values, label='Actual TSLA Price', color='black', linewidth=2)
plt.plot(y_test.index, lr_preds, label=f'Linear Regression (R²: {lr_r2:.4f})', color='#E63946', linestyle='--')
plt.plot(y_test.index, tuned_rf_preds, label=f'Optimized Random Forest (R²: {tuned_rf_r2:.4f})', color='#457B9D', linestyle='--')

plt.title('Final Model Comparison: TSLA Stock Price Prediction', fontsize=16, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Price')
```

```
plt.xlabel('Date', fontsize=12)  
plt.ylabel('Stock Price (USD)', fontsize=12)  
plt.legend(fontsize=12, loc='upper left')  
plt.grid(True, linestyle=':', alpha=0.5)  
  
plt.tight_layout()  
plt.show()
```

